

فاز چهارم • معماری نرم افزار و مقایسه رویکردها

# سامانه یکپارچه آموزش آنلاین مدارس

پلتفرم «شبکه شاد» — معماری سیستم، سرویس گرایی و زیرساخت ابری

---

نمودار معماری • سرویس ها • ADR • استقرار ابری

## نقشه راه ارائه

۱ معماری کلی سیستم و لایه‌ها

۱

۲ مقایسه رویکرد سنتی و مدرن توسعه

۲

۳ مقایسه سبک‌های معماری (تک‌هسته‌ای / لایه‌ای / سرویس‌گرا)

۳

۴ زیرساخت Cloud Native

۴

۵ شناسایی سرویس‌های SOA

۵

۶ رابط سرویس‌ها و قراردادهای داده‌ای

۶

۷ معماری ترکیبی نهایی و تصمیمات (ADR)

۷

۸ نمودار مؤلفه و نمودار استقرار

۸

۹ مدیریت نیازمندی‌ها

۹

۱۰ سناریوی همکاری سرویس‌ها و جمع‌بندی

۱۰

## لایه‌های معماری «شاد»

معماری اجتماعی-فناورانه و ترکیبی (Hybrid) که از چند الگوی معماری برای پشتیبانی از میلیون‌ها کاربر همزمان استفاده می‌کند.

1

### لایه کلاینت

اپلیکیشن دانش‌آموز، اپلیکیشن معلم و نسخه وب — ارسال درخواست و نمایش نتایج

2

### لایه سرویس‌ها

احراز هویت، مدیریت کلاس، محتوا، پیام‌رسانی، اعلان‌ها و آزمون‌ها

3

### لایه داده

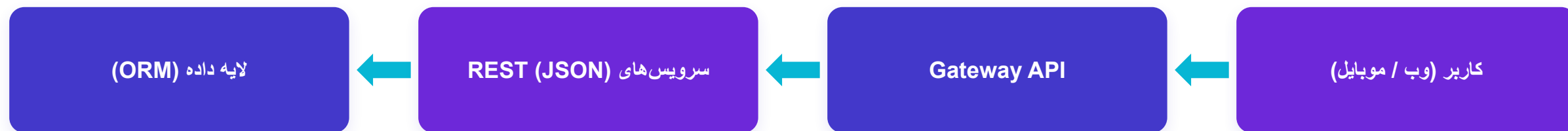
پایگاه‌داده کاربران، کلاس‌ها، نمرات و تاریخچه پیام‌ها

4

### لایه زیرساخت

سرورهای مرکزی، شبکه، CDN و سیستم‌های امنیت و مانیتورینگ

## ارتباط چندلایه مبتنی بر API



فایل‌های حجیم (ویدیو، PDF) از طریق CDN توزیع می‌شوند و اعلان‌ها/پیام‌ها به صورت Real-Time منتقل می‌شوند.

1 کلاینت با سرور از طریق HTTPS و Gateway API ارتباط برقرار می‌کند.

2 سرویس‌ها مستقل اما از طریق REST API با یکدیگر تعامل دارند؛ تبادل داده با فرمت JSON.

3 تمام سرویس‌ها برای ذخیره و بازیابی اطلاعات به لایه داده متصل‌اند.

## چرا Agile + Scrum + DevOps؟

### مدل آبشاری (Waterfall)

- مراحل خطی: تحلیل، طراحی، پیاده‌سازی، آزمون، استقرار
- مستندسازی دقیق و کنترل مناسب فرآیند
- تغییرات پرهزینه و زمان‌بر
- نیازمندی‌های سامانه آموزشی ثابت نمی‌مانند
- ناسازگار با تغییر مداوم قوانین آموزشی و نیاز کاربران

### چابکی (Agile/Scrum) + DevOps

- توسعه در Sprint های کوتاه با خروجی قابل استفاده
- واکنش سریع به امکانات جدید و تغییر نیازمندی‌ها
- یکپارچگی تیم توسعه و عملیات: DevOps
- خودکارسازی Build / Test / Deploy
- کاهش خطا و افزایش سرعت استقرار نسخه‌های جدید

## تک‌هسته‌ای در برابر لایه‌ای در برابر سرویس‌گرا

| معیار              | تک‌هسته‌ای (Monolithic)      | لایه‌ای (Layered)                 | سرویس‌گرا (SOA)               |
|--------------------|------------------------------|-----------------------------------|-------------------------------|
| مقیاس‌پذیری        | کل سیستم باید مقیاس داده شود | کل سیستم به‌صورت واحد اجرا می‌شود | هر سرویس مستقل مقیاس‌پذیر است |
| توسعه موازی تیم‌ها | عملاً غیرممکن                | محدود به‌دلیل وابستگی لایه‌ها     | کاملاً امکان‌پذیر             |
| وابستگی اجزا       | بسیار بالا                   | بالا بین لایه‌ها                  | پایین (Loose Coupling)        |
| مناسب برای         | سامانه‌های کوچک              | پروژه‌های کوچک تا متوسط           | سامانه‌های بزرگ و پرتراфик    |

نتیجه: معماری سرویس‌گرا به‌دلیل استقلال سرویس‌ها و مقیاس‌پذیری هدفمند، انتخاب اصلی برای «شاد» است.

# چرا Cloud Native؟

## استقرار مستقل

تغییر قوانین در یک سرویس نیازمند استقرار مجدد کل سامانه نیست.

## دسترسی پذیری بالا

اجرای چند نمونه از هر سرویس؛ خرابی یک نمونه اختلالی ایجاد نمی‌کند.

## مقیاس دهی هدفمند

فقط سرویس پرتراфик (مثل آزمون آنلاین) مقیاس داده می‌شود، نه کل سامانه.

## آماده برای توسعه آینده

افزودن سرویس‌های جدید بدون تغییر اساسی در سایر بخش‌ها.

## استانداردسازی محیط

کانتینرها و ارکستریشن، اختلاف محیط توسعه/تست/تولید را حذف می‌کنند.

## بهینه‌سازی هزینه

پرداخت متناسب با مصرف واقعی، به جای تهیه سخت‌افزار با ظرفیت حداکثری.

## ۱۰ سرویس اصلی بر اساس اصول SOA

1

احراز هویت و مدیریت کاربران

2

مدیریت مدارس، کلاس‌ها و دروس

3

کلاس آنلاین

4

پیام‌رسان

5

مدیریت تکالیف

6

آزمون و ارزشیابی

7

حضور و غیاب

8

مدیریت فایل

9

اعلان‌ها

10

گزارش‌گیری و تحلیل اطلاعات

اصل حاکم: Separation of Concerns — هر سرویس فقط مسئول یک دسته وظیفه مشخص است و پایگاه‌داده اختصاصی خود را دارد.



## نمونه: Authentication Service

### عملیات سرویس (Service Interface)

```
login(username, password) → Authentication  
Token  
logout(token)  
register(userInformation)  
verifyToken(token)
```

### قرارداد دادهای User — (Data Contract)

| فیلد        | نوع داده |
|-------------|----------|
| userId      | Integer  |
| name        | String   |
| email       | String   |
| password    | String   |
| role        | String   |
| phoneNumber | String   |

## معماری ترکیبی (Hybrid Architecture)

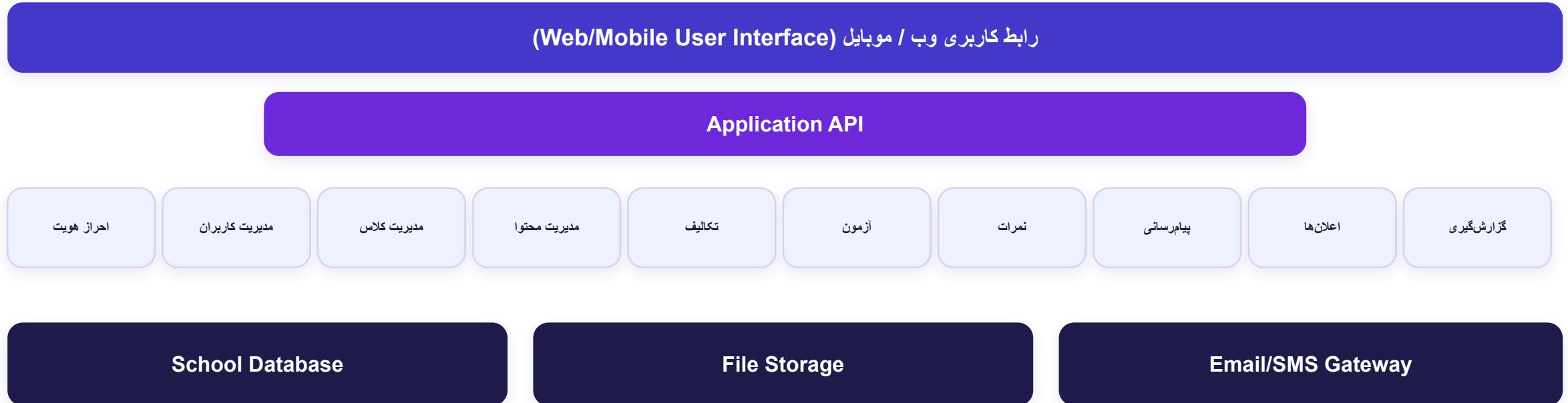


هر لایه نقاط قوت خود را به سیستم می‌افزاید: امنیت متمرکز (کلاينت-سرور)، نگهداری ساده (لایه‌ای)، استقلال توسعه (SOA) و پاسخگویی لحظه‌ای (رویدادمحور).

## خلاصه ADRها

| کد     | تصمیم                         | توجیه کلیدی                                  |
|--------|-------------------------------|----------------------------------------------|
| ADR-01 | (Hybrid) معماری ترکیبی        | و رویدادمحور SOA، ترکیب کلاینت-سرور، لایه‌ای |
| ADR-02 | RESTful API                   | JSON سادگی، سازگاری بالا و تبادل داده با     |
| ADR-03 | سرویس متمرکز احراز هویت       | یکپارچگی امنیت و جلوگیری از تکرار منطق ورود  |
| ADR-04 | Database per Service          | استقلال داده و کاهش وابستگی بین سرویس‌ها     |
| ADR-05 | Event Bus + معماری رویدادمحور | کاهش وابستگی مستقیم و پاسخ‌گویی لحظه‌ای      |
| ADR-06 | استقرار روی زیرساخت ابری      | مقیاس‌پذیری خودکار و دسترس‌پذیری بالا        |

# ساختار ماژولار سامانه



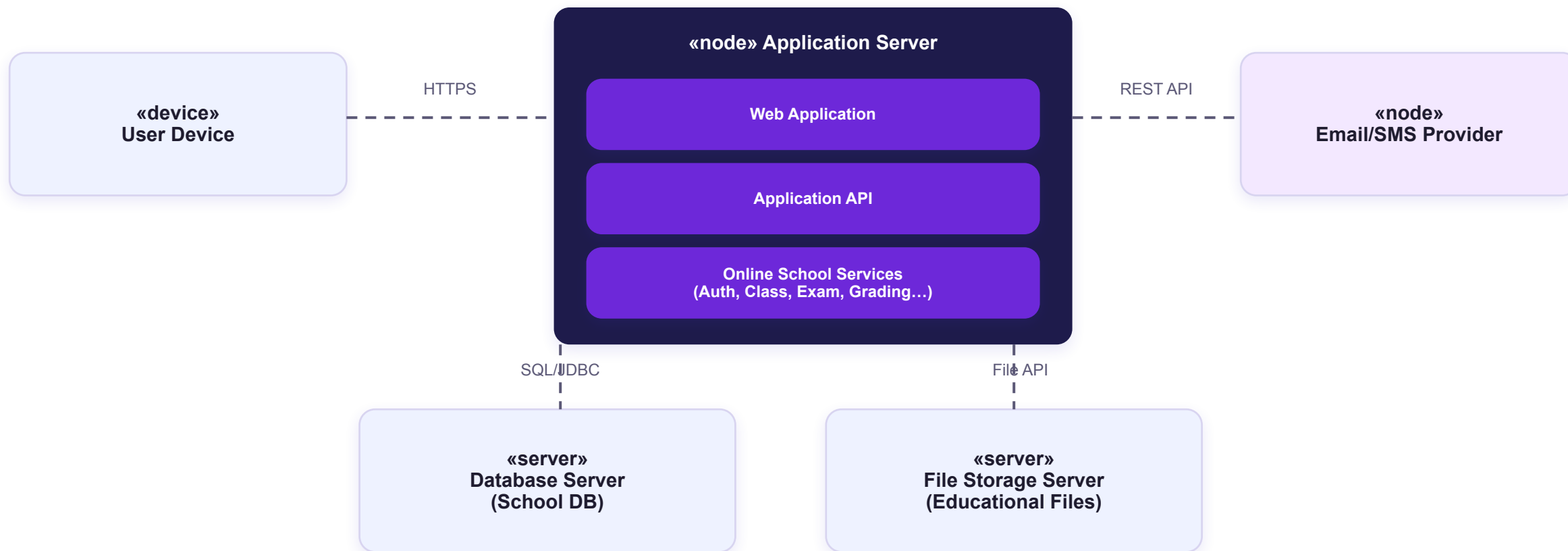
ارتباط تمام مؤلفه‌ها از طریق رابط‌های استاندارد (Interfaces) انجام می‌شود تا وابستگی مستقیم اجزا کاهش یابد.

مدیریت تکالیف درسی — ارسال توسط معلم، تحویل توسط دانش‌آموز — Assignment Service

خارجی Email/SMS Gateway ارسال اعلان از طریق — Notification Service

تولید گزارش‌های آماری از عملکرد دانش‌آموزان و کلاس‌ها — Reporting Service

## معماری فیزیکی و توزیع روی سرورها



جداسازی مسئولیت‌ها: سرور برنامه (منطق) ، سرور پایگاه‌داده (داده ساختاری) ، سرور فایل (محتوای حجیم) و سرویس خارجی پیام‌رسانی.

## رویکرد سنتی در برابر رویکرد مدرن

### رویکرد سنتی

- نیازمندی‌ها در ابتدای پروژه کامل مستند می‌شوند
- هر تغییر نیازمند بازنگری رسمی اسناد و طراحی
- فاصله زیاد بین سند SRS و نیاز واقعی کاربران
- افزایش هزینه و تأخیر در تحویل پروژه

### رویکرد مدرن (Agile)

- نیازمندی‌ها به تدریج و بر اساس بازخورد تکامل می‌یابند
- تغییرات بخش طبیعی فرآیند توسعه هستند
- قابلیت ردیابی بالاتر بین نیاز، طراحی و آزمون
- هماهنگی مستمر با نیاز واقعی ذی‌نفعان (دانش‌آموز، معلم، والدین)

## سناریو: برگزاری یک کلاس آنلاین



استقلال سرویس‌ها + ارتباط رویدادمحور → در صورت اختلال یک سرویس، سایر سرویس‌ها همچنان فعال باقی می‌مانند.

# جمع‌بندی نهایی

1

معماری ترکیبی (کلاينت-سرور + لایه‌ای + SOA + رویدادمحور) انتخاب اصلی سامانه است

2

توسعه بر پایه Agile، Scrum و DevOps برای سازگاری با تغییرات مداوم آموزشی

3

استقرار روی زیرساخت ابری (Cloud Native) برای مقیاس‌پذیری و دسترس‌پذیری بالا

4

۱۰ سرویس مستقل با پایگاه‌داده اختصاصی، مطابق اصول SOA

نتیجه: سامانه‌ای پایدار، مقیاس‌پذیر و آماده برای توسعه‌های آینده آموزش الکترونیکی